



Express-MongoDB Integration

Building Production-Ready APIs

Chapter Overview

- 01** MongoDB Fundamentals
- 02** Mongoose Models and Schemas
- 03** Express + MongoDB CRUD API
- 04** Error Handling and Validation
- 05** Authentication with JWT
- 06** Security and Production Practices



MongoDB Fundamentals

Section 1



What is MongoDB?

Document Database

- Stores data as BSON (Binary JSON) documents — flexible, schema-optional
- Documents live in collections (like tables), databases group collections
- Each document can have a different structure — no rigid schema required

Key Concepts

- Document — a single record (JSON-like object with `_id`)
- Collection — a group of documents (like a table)
- Database — a group of collections
- BSON — Binary JSON with extra types: ObjectId, Date, Decimal128



Native Driver — Connect and CRUD

```
import { MongoClient } from 'mongodb';

const client = new MongoClient(process.env.MONGODB_URI);
await client.connect();

const db = client.db('app');
const collection = db.collection('books');

// Create
await collection.insertOne({ title: 'Test', author: 'Jane' });

// Read
const books = await collection.find().toArray();
const one = await collection.findOne({ title: 'Test' });

// Update
await collection.updateOne(
  { title: 'Test' },
  { $set: { pages: 200 } }
);

// Delete
await collection.deleteOne({ title: 'Test' });
```



Native Driver Trade-offs

Advantages

- Full control over queries — no abstraction layer between you and MongoDB
- Better performance for complex aggregation pipelines
- Smaller dependency footprint — just the mongodb package

Drawbacks

- No schema enforcement — you must validate all input manually
- No middleware hooks — pre/post-save logic must be coded by hand
- More boilerplate for common patterns like CRUD operations

Why Mongoose?

- Mongoose adds schemas, validation, middleware, and virtual properties on top



Native Driver vs Mongoose

Native MongoDB Driver vs Mongoose

Native MongoDB Driver

```
import {{ MongoClient }}
from 'mongodb';
const client =
  new MongoClient(uri);
await client.connect();
```

- > ✓ Full MongoDB control
- > ✓ Best for aggregations
- > ✓ Lower overhead
- > ✗ No schema enforcement
- > ✗ Manual validation
- > ✗ More boilerplate

Use Native Driver When:

High-perf systems, complex aggregations, full MongoDB control needed

Mongoose ODM

```
import mongoose
from 'mongoose';
await mongoose.connect(
  uri, {{ maxPoolSize: 10 }});
```

- > ✓ Schema + validation
- > ✓ Middleware hooks
- > ✓ Less boilerplate
- > ✗ Abstraction overhead
- > ✗ Learning curve
- > ✗ Less aggregation control

Use Mongoose When:

Structured CRUD apps, need validation + hooks, rapid development



Mongoose Models and Schemas

Section 2



Why Mongoose?

First Understand the Native Driver

- Mongoose is not the starting point — first learn plain MongoDB with the native driver
- Understand collections, documents, and manual CRUD before adding ODM abstraction

Object Document Mapper (ODM)

- Mongoose adds structure to MongoDB — schemas, validation, middleware
- Much less boilerplate than the raw MongoDB driver for CRUD operations

Key Features

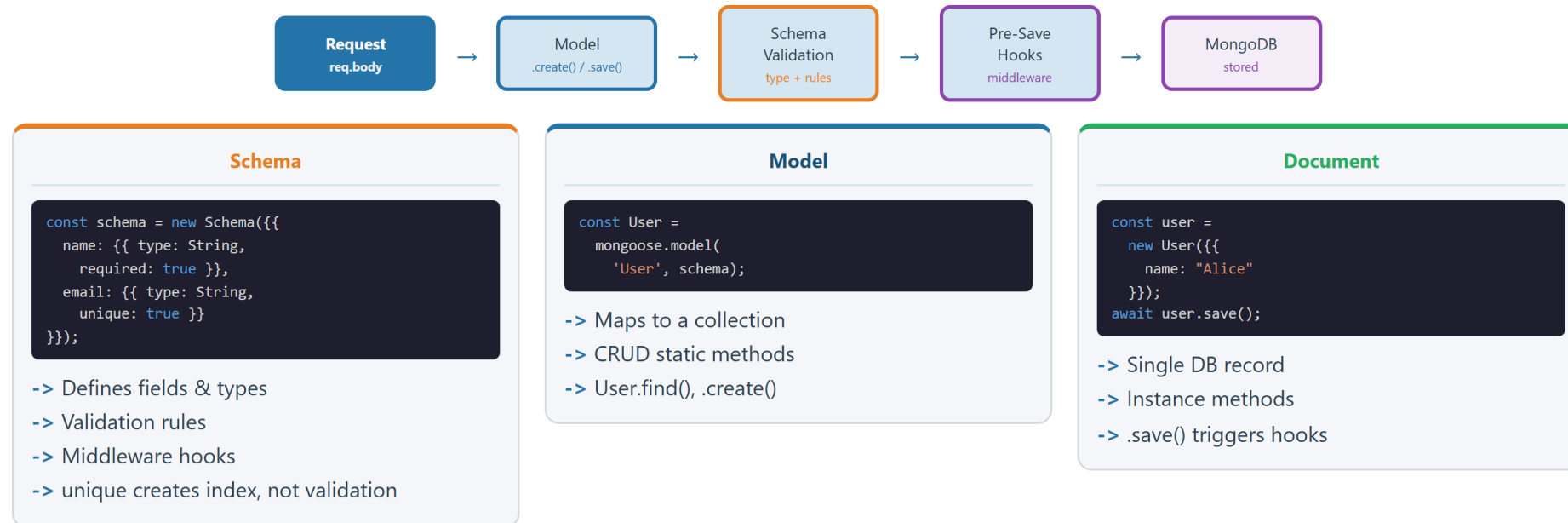
- Schema definitions with type checking and required fields
- Built-in validation — min/max, regex, enum, custom validators
- Middleware hooks — run logic before/after save, validate, deleteOne



Schema, Model, and Document

Mongoose: Schema → Model → Document

Data Flow: Request to Database



Request → Model → Schema Validation → Pre-Save Hooks → MongoDB



Defining a Mongoose Schema

```
import mongoose from 'mongoose';

const bookSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  author: { type: String, required: true, index: true },
  pages: { type: Number, min: 1 },
  isbn: { type: String, unique: true }, // creates index, not validation
  genre: { type: String, enum: ['fiction', 'non-fiction', 'technical'] },
  published: { type: Date, default: Date.now },
}, { timestamps: true }); // adds createdAt, updatedAt

// Note: unique: true creates a unique INDEX – it does NOT add a
// Mongoose validator. Duplicate key errors come from MongoDB (code 11000),
// not from Mongoose validation.

export default mongoose.model('Book', bookSchema);
```

Connecting to MongoDB

```
import mongoose from 'mongoose';

async function connectDB(retries = 5) {
  for (let i = 0; i < retries; i++) {
    try {
      await mongoose.connect(process.env.MONGODB_URI, {
        maxPoolSize: 10,
        serverSelectionTimeoutMS: 5000,
      });
      console.log('Connected to MongoDB');
      return;
    } catch (err) {
      console.error(`Attempt ${i + 1} failed:`, err.message);
      if (i < retries - 1) await new Promise(r => setTimeout(r, 2000));
    }
  }
  process.exit(1);
}

// Connection event logging
mongoose.connection.on('disconnected', () => console.warn('DB disconnected'));
mongoose.connection.on('reconnected', () => console.log('DB reconnected'));

// Graceful shutdown on SIGINT and SIGTERM
for (const sig of ['SIGINT', 'SIGTERM']) {
  process.on(sig, async () => {
    await mongoose.connection.close();
    process.exit(0);
  });
}

export default connectDB;
```

Schema Features

Virtual Properties

- Computed fields derived from stored data — e.g. fullName from first + last
- Not persisted to MongoDB — calculated on access

Middleware Hooks

- pre('save') — run logic before persisting (e.g. hash passwords)
- post('save') — run logic after persisting (e.g. send welcome email)

Important Gotcha

- findByIdAndUpdate does NOT trigger pre-save middleware
- Use find + modify + save() when middleware matters (e.g. password hashing)



Schema Middleware and Methods

```
// Virtual property
bookSchema.virtual('summary').get(function () {
  return `${this.title} by ${this.author}`;
});

// Pre-save middleware
bookSchema.pre('save', function (next) {
  this.title = this.title.trim();
  next();
});

// Instance method
bookSchema.methods.isRecent = function () {
  const oneYear = 365 * 24 * 60 * 60 * 1000;
  return Date.now() - this.published < oneYear;
};

// Static method
bookSchema.statics.findByGenre = function (genre) {
  return this.find({ genre });
};
```



Express + MongoDB CRUD API

Section 3



Project Architecture

Layered Separation of Concerns

- routes/ — URL-to-handler mapping, no business logic
- controllers/ — business logic, calls model methods, sends responses
- models/ — Mongoose schemas and database operations
- middleware/ — reusable functions: auth, validation, error handling

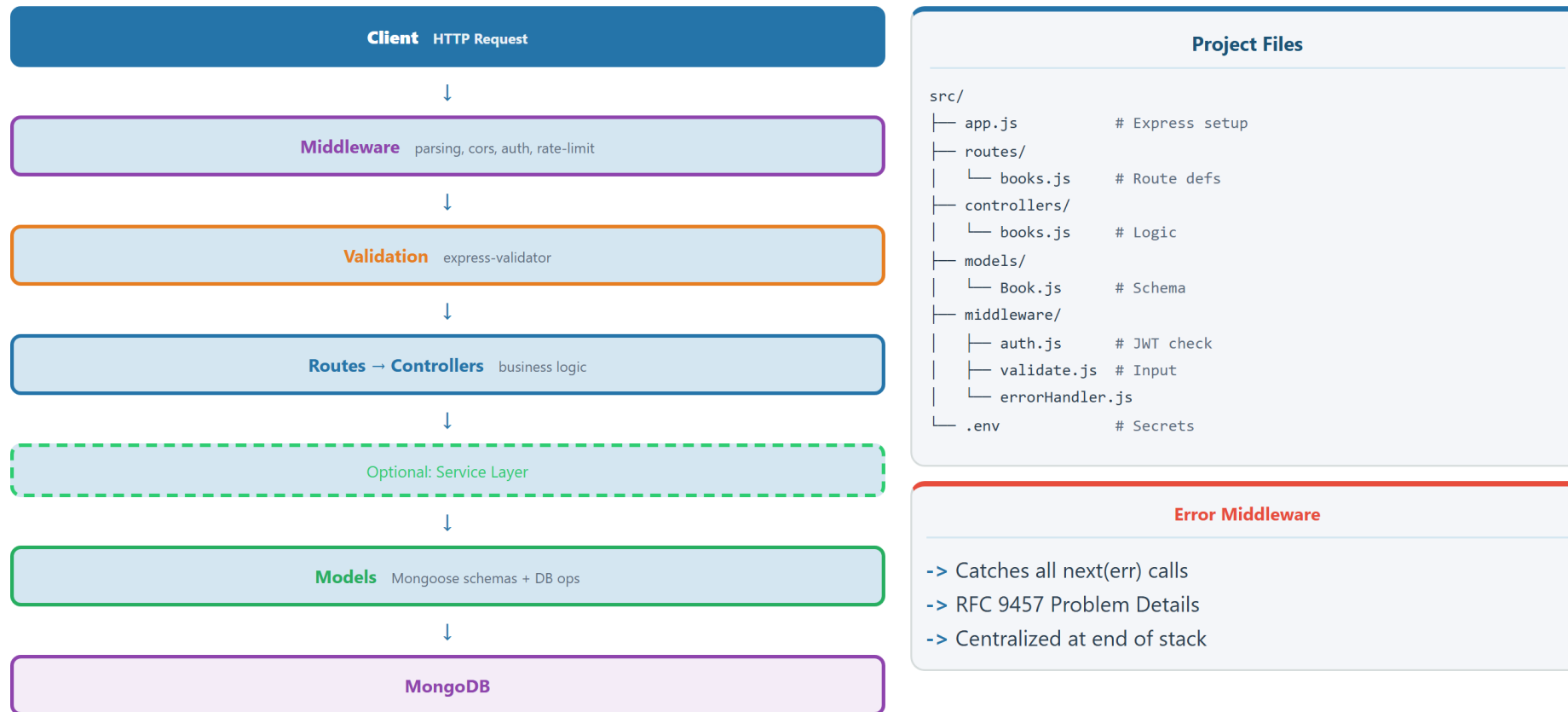
Environment Variables

- Use dotenv to load .env files — never hardcode connection strings or secrets
- Separate .env.development and .env.production for each environment



CRUD API Architecture

Express CRUD — Layered Architecture



Express App Setup

```
import 'dotenv/config'; // load .env before anything else
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import connectDB from './db.js';
import bookRoutes from './routes/books.js';
import { errorHandler } from './middleware/errorHandler.js';

const app = express();

// Security middleware – order matters
app.use(helmet());
app.use(cors({
  origin: process.env.CORS_ORIGIN || 'http://localhost:5173',
  credentials: true,
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Routes
app.use('/api/books', bookRoutes);

// Centralized error handler MUST be registered after all routes
app.use(errorHandler);

// Start server
connectDB().then(() => {
  app.listen(process.env.PORT || 3000);
});
```



asyncHandler — Eliminate try/catch

```
// Async wrapper – wraps every route handler automatically
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// BEFORE: repetitive try/catch in every route
router.get('/', async (req, res, next) => {
  try {
    const books = await Book.find();
    res.json(books);
  } catch (err) { next(err); }
});

// AFTER: clean, no try/catch needed
router.get('/', asyncHandler(async (req, res) => {
  const books = await Book.find().select('-__v');
  res.json(books);
}));
```

GET Routes — List and Find by ID

```
import express from 'express';
import Book from '../models/Book.js';

const router = express.Router();

// GET all books
router.get('/', asyncHandler(async (req, res) => {
  const books = await Book.find().select('-__v'); // projection, not security
  res.json(books);
}));

// GET book by ID
// Note: findById() throws CastError for malformed ObjectIds
// which is caught by asyncHandler and sent to error middleware
router.get('/:id', asyncHandler(async (req, res) => {
  const book = await Book.findById(req.params.id);
  if (!book) return res.status(404).json({ error: 'Book not found' });
  res.json(book);
}));
```



POST — Create a New Resource

```
// POST create a book
// Explicit field mapping prevents mass-assignment vulnerabilities
// Never pass req.body directly to Model.create()
router.post('/', validateBook, handleValidation,
  asyncHandler(async (req, res) => {
    const { title, author, pages, isbn, genre } = req.body;
    const book = await Book.create({
      title, author, pages, isbn, genre,
    });
    res.status(201).json(book);
  })
);

// Validation errors from express-validator are caught by
// handleValidation middleware BEFORE reaching this handler.
// Mongoose validation errors (schema rules) are caught by
// asyncHandler and forwarded to centralized error middleware.
```

PUT Update and DELETE

```
// PUT update a book
router.put('/:id', validateBook, handleValidation,
  asyncHandler(async (req, res) => {
    const book = await Book.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    // Note: findByIdAndUpdate does NOT trigger pre-save hooks
    if (!book) return res.status(404).json({ error: 'Not found' });
    res.json(book);
  })
);

// DELETE a book
router.delete('/:id', asyncHandler(async (req, res) => {
  const book = await Book.findByIdAndDelete(req.params.id);
  if (!book) return res.status(404).json({ error: 'Not found' });
  res.status(204).end();
}));
```



Safe Update Patterns

findByIdAndUpdate Options

- { new: true } — return the updated document instead of the original
- { runValidators: true } — enforce schema rules on the update
- Without runValidators, updates bypass all schema validation

findByIdAndUpdate vs save()

- findByIdAndUpdate — atomic, single DB call, skips pre-save middleware
- find + modify + save() — triggers pre-save hooks, more flexible
- Use save() when middleware matters (e.g. password hashing)



Error Handling and Validation

Section 4



Dual-Layer Validation Strategy

API Layer — express-validator

- First line of defense: validate request shape before hitting the controller
- Declarative rules: `body('email').isEmail()`, `body('password').isLength({ min: 8 })`
- Catches malformed input early — reduces unnecessary database calls

Schema Layer — Mongoose

- Second line of defense: enforces data integrity rules at the model level
- Required fields, type checking, enum constraints, custom validators

Why Both?

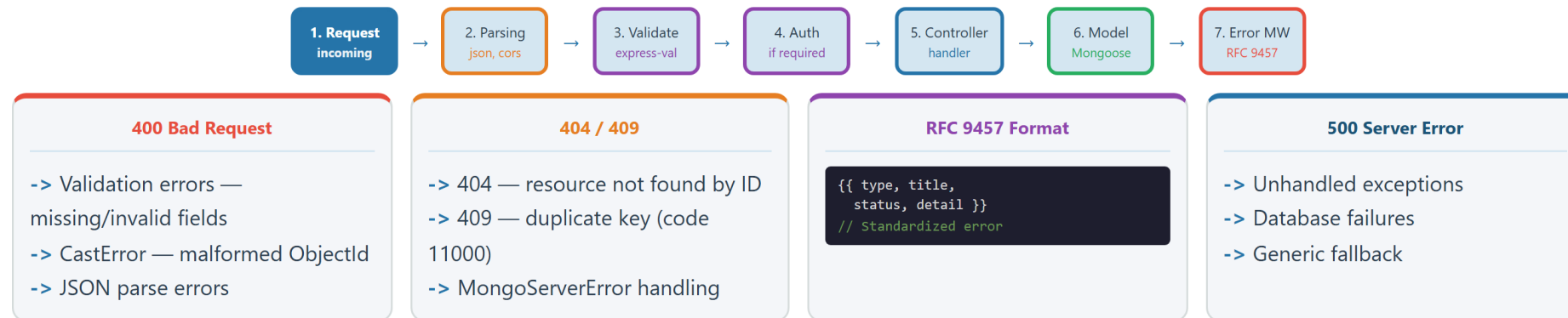
- Defense-in-depth — API validation can be bypassed if models are called directly



Validation and Error Pipeline

Validation & Error Handling Pipeline

Correct Pipeline Order



Validate early (express-validator) → Mongoose schema rules → centralized error middleware with RFC 9457 Problem Details (application/problem+json)

Validation Middleware

```
import { body, validationResult } from 'express-validator';

const validateBook = [
  body('title').trim().notEmpty().withMessage('Title is required'),
  body('author').trim().notEmpty().withMessage('Author is required'),
  body('pages').optional().isInt({ min: 1 }),
  body('isbn').optional().isISBN(),
];

// Reusable validation check middleware
const handleValidation = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  next();
};

// Wire into route
router.post('/', validateBook, handleValidation, asyncHandler(createBook));
```

Centralized Error Middleware — RFC 9457

```
// Centralized RFC 9457 Problem Details error handler
// Responds with application/problem+json format
// Must have 4 parameters so Express recognizes it as error MW
app.use((err, req, res, next) => {
  if (err.name === 'ValidationError') {
    return res.status(400).json({
      type: 'https://api.example.com/errors/validation',
      title: 'Validation Failed',
      status: 400,
      detail: Object.values(err.errors).map(e => e.message).join(', '),
    });
  }
  if (err.name === 'CastError') {
    return res.status(400).json({
      type: 'https://api.example.com/errors/invalid-id',
      title: 'Invalid ID Format', status: 400,
    });
  }
  if (err.code === 11000) {
    return res.status(409).json({
      type: 'https://api.example.com/errors/duplicate',
      title: 'Duplicate Key', status: 409,
    });
  }
  console.error(err.stack);
  res.status(500).json({
    type: 'https://api.example.com/errors/server',
    title: 'Internal Server Error', status: 500,
  });
});
```



NoSQL Injection Protection

The Threat

- Attackers can pass objects like { \$gt: " } in query parameters
- This bypasses authentication checks or returns unauthorized data

Defenses

- `mongoose.set('strictQuery', true)` — ignores unknown query fields
- `express-validator` sanitization — strip \$ operators from input
- Explicit field mapping in create/update — never pass `req.body` directly to Model
- Type checking at the API layer — reject objects where strings are expected



Authentication with JWT

Section 5



Password Security

Never Store Plain-Text Passwords

- Use bcrypt to hash passwords — one-way transformation, cannot be reversed
- bcrypt adds a random salt — same password produces different hashes each time

How bcrypt Works

- Register: hash with `bcrypt.hash(password, saltRounds)` before saving
- Login: compare with `bcrypt.compare(plainText, storedHash)`
- Salt rounds (cost factor): 10-12 recommended — higher is slower but more secure

For New Systems

- Consider Argon2 — winner of the Password Hashing Competition, more memory-hard



User Model with Pre-Save Hook

```
import mongoose from 'mongoose';
import bcrypt from 'bcrypt';

const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 8 },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
}, { timestamps: true });

// Hash password before saving – only if modified
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

// Instance method for password comparison
userSchema.methods.comparePassword = async function (candidate) {
  return bcrypt.compare(candidate, this.password);
};

export default mongoose.model('User', userSchema);
```

JWT Authentication Flow

What is a JWT?

- JSON Web Token — a signed, base64-encoded string: header.payload.signature
- Stateless — the server does not store sessions, the token carries identity

The Flow

- 1. Client sends credentials (email + password) to POST /login
- 2. Server verifies credentials, signs access token (15m) + refresh token (7d)
- 3. Client stores tokens and sends access token in Authorization header
- 4. Server middleware verifies token: expiration → signature → role extraction



JWT Auth Sequence

JWT Authentication Flow



Short-lived access tokens + long-lived refresh tokens with rotation — revoke on logout or compromise



Registration Route

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';

// POST /api/auth/register
router.post('/register', asyncHandler(async (req, res) => {
  const { email, password } = req.body;

  const existing = await User.findOne({ email });
  if (existing) {
    return res.status(409).json({ error: 'Email already registered' });
  }

  const user = await User.create({ email, password });
  // password is hashed by pre-save hook

  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '15m', algorithm: 'HS256' }
  );
  res.status(201).json({ token });
}));
```



Login Route

```
// POST /api/auth/login
router.post('/login', asyncHandler(async (req, res) => {
  const { email, password } = req.body;

  // 1. Find user by email
  const user = await User.findOne({ email });
  if (!user) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  // 2. Compare password using instance method
  const match = await user.comparePassword(password);
  if (!match) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  // 3. Sign access + refresh tokens
  const accessToken = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '15m', algorithm: 'HS256' }
  );
  const refreshToken = jwt.sign(
    { id: user._id },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: '7d' }
  );
  res.json({ accessToken, refreshToken });
}));
```



Auth Middleware — Stronger Verification

```
import jwt from 'jsonwebtoken';

function authenticate(req, res, next) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = header.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET, {
      algorithms: ['HS256'],
      audience: process.env.JWT_AUDIENCE || 'your-app',
      issuer: process.env.JWT_ISSUER || 'your-auth-server',
    });
    req.user = decoded; // { id, role, iat, exp }
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```



Protecting Routes

Authentication vs Authorization

- Authentication — who are you? Verify identity with JWT
- Authorization — what can you do? Check roles or ownership

Applying Auth Middleware

- Per-route: `router.get('/profile', authenticate, getProfile)`
- Per-router: `router.use(authenticate)` — protects all routes below

Ownership Checks

- Users should only modify their own resources — check `req.user.id === resource.owner`
- Admin routes need a separate role check middleware



Role-Based Authorization

```
// Role-checking middleware factory
function authorize(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }
    next();
  };
}

// Usage – only admins can delete users
router.delete(
  '/users/:id',
  authenticate,
  authorize('admin'),
  asyncHandler(deleteUser)
);

// Public routes (no middleware)
router.get('/books', asyncHandler(listBooks));
```



JWT Security Best Practices

Token Expiration

- Access tokens: 15 minutes — short-lived, limits damage if compromised
- Refresh tokens: 7 days — used only to issue new access tokens

Token Rotation and Revocation

- When refreshing, invalidate the old refresh token and issue a new pair
- On logout, revoke refresh token (store revoked tokens in DB or Redis)
- If a revoked token is reused, invalidate ALL tokens for that user

Secret Management

- JWT_SECRET must be a long, random string — at least 256 bits
- Store in environment variables, never in source code

Client-Side Storage

- httpOnly cookies are safer than localStorage — immune to XSS attacks
-  – localStorage is simpler but accessible to any JavaScript on the page

Security and Production Practices

Section 6



Helmet — Secure HTTP Headers

```
import helmet from 'helmet';

// helmet() sets many security headers in one call:
// - Content-Security-Policy: restricts resource loading
// - X-Content-Type-Options: nosniff – prevents MIME sniffing
// - Strict-Transport-Security: enforces HTTPS
// - X-Frame-Options: DENY – prevents clickjacking
// - X-XSS-Protection: 0 – disables broken legacy filter

app.use(helmet());

// Always add helmet BEFORE routes – it sets response headers
// that apply to all downstream responses.
```

Rate Limiting

```
import rateLimit from 'express-rate-limit';

// Protect auth endpoints from brute-force attacks
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15-minute window
  max: 100,                 // max 100 requests per window
  standardHeaders: true,    // Return rate limit info in headers
  legacyHeaders: false,
  message: {
    type: 'https://api.example.com/errors/rate-limit',
    title: 'Too Many Requests',
    status: 429,
  },
});

app.use('/api/auth', authLimiter);

// Consider stricter limits for login: max 5 per 15 min
// Consider lighter limits for read endpoints: max 1000 per 15 min
```



CORS Configuration

```
import cors from 'cors';

// BAD: allows any origin
// app.use(cors());

// GOOD: restrict to your frontend
app.use(cors({
  origin: process.env.CORS_ORIGIN || 'https://yourfrontend.com',
  credentials: true, // allow cookies/auth headers
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));

// In production, CORS_ORIGIN should be set to your actual domain.
// Never use '*' with credentials: true – browsers will reject it.
```



Observability

Logging — pino or winston

- Structured JSON logs with request ID, timestamp, level
- pino is faster (low-overhead), winston is more configurable
- Log requests, responses, errors — never log passwords or tokens

Metrics — Prometheus

- Expose /metrics endpoint with prom-client for request counts, latency, error rates
- Set up alerts on error rate spikes and high latency percentiles (p95, p99)

Tracing — OpenTelemetry

- Distributed tracing across services — tracks a request through all microservices
- auto-instrument Express, Mongoose, and HTTP with @opentelemetry/auto-instrumentations-node



Environment Separation

Separate Configuration Per Environment

- Use `.env.development` for local settings (localhost URLs, debug logging)
- Use `.env.production` for deployed settings (real URLs, error-only logging)
- Load with dotenv: `dotenv.config({ path: `.env.${NODE_ENV}` })`

What to Separate

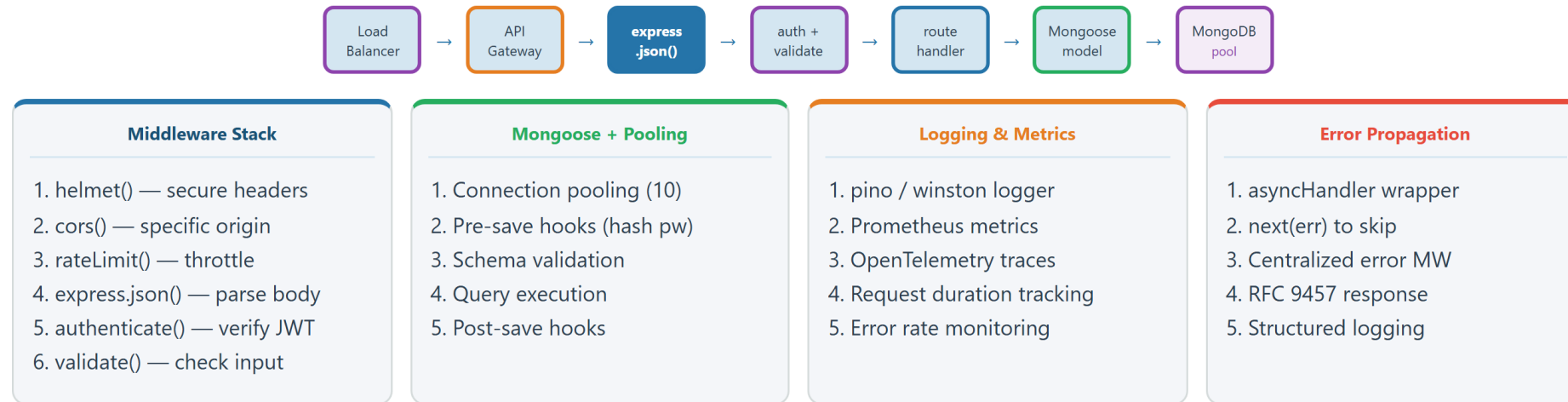
- `MONGODB_URI` — local vs Atlas cluster connection string
- `CORS_ORIGIN` — `http://localhost:5173` vs `https://yourfrontend.com`
- `JWT_SECRET` — different secrets per environment
- `LOG_LEVEL` — 'debug' in dev, 'error' in production



Full Request Lifecycle

Production Request Lifecycle

Production Request Lifecycle



Load Balancer → API Gateway → Express middleware → Mongoose (pooled) → MongoDB — with observability at every layer



Pagination — Offset vs Cursor

Offset-Based (Simple)

- Use `.skip()` and `.limit()` — easy to implement, familiar page numbers
- Problem: skip becomes slow on large datasets (scans and discards rows)

Cursor-Based (Performant)

- Use `_id` comparison: `Book.find({ _id: { $gt: lastId } }).limit(20)`
- Uses indexes — no scanning, consistently fast even on millions of documents
- Client sends the last `_id` from the previous page as the cursor
- Trade-off: no arbitrary page jumps — clients navigate forward/backward only



Cursor-Based Pagination

```
// Cursor-based pagination – efficient for large datasets
router.get('/', asyncHandler(async (req, res) => {
  const limit = Math.min(parseInt(req.query.limit) || 20, 100);
  const cursor = req.query.cursor; // last _id from previous page

  const query = cursor
    ? { _id: { $gt: cursor } }
    : {};

  const books = await Book.find(query)
    .select('title author published')
    .sort({ _id: 1 })
    .limit(limit + 1); // fetch 1 extra to check if more exist

  const hasMore = books.length > limit;
  if (hasMore) books.pop(); // remove the extra

  const nextCursor = hasMore ? books[books.length - 1]._id : null;
  res.json({ books, nextCursor, hasMore });
}));
```



When NOT to Use Mongoose

High-Performance Systems

- Native driver has less overhead — no schema validation, no middleware hooks
- For bulk operations (10k+ docs): native insertMany is significantly faster

Complex Aggregations

- Mongoose aggregate() works but the native pipeline API is more natural
- Advanced features (\$graphLookup, \$merge) are easier with the native driver

Need Full MongoDB Control

- Change streams, transactions across collections, custom write concerns
- When you need to use the latest MongoDB features before Mongoose supports them



Key Takeaways

Key Takeaways

1. Understand MongoDB with the native driver before introducing Mongoose abstractions
2. Use ESM imports and asyncHandler to eliminate boilerplate try/catch blocks
3. Validate at both API (express-validator) and schema (Mongoose) layers — defense in depth
4. Standardize error responses with RFC 9457 Problem Details (application/problem+json)
5. Short-lived access tokens + refresh token rotation for secure JWT auth
6. Production: helmet, CORS, rate limiting, connection pooling, and observability

